

Automated Detection of Hardcoded Secrets in Modern Source Code Repositories using LLM-Augmented Static Analysis

A Project Report

Submitted by:

Satyajeet Satapathy	2241019340
Dinanath Dash	2241004161
Avignyan Sarangi	2241019304
Ashutosh Raj	2241018015

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**Faculty of Engineering and Technology, Institute of Technical Education and
Research**

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY

Bhubaneswar, Odisha, India

(June 2026)



CERTIFICATE

This is to certify that the project report titled “Automated Detection of Hardcoded Secrets in Modern Source Code Repositories using LLM-Augmented Static Analysis” being submitted by Satyajeet Satapathy, Dinanath Dash, Avignyan Sarangi and Ashutosh Raj of Section-2241026 to the Institute of Technical Education and Research, Siksha ‘O’ Anusandhan (Deemed to be) University, Bhubaneswar for the partial fulfillment for the degree of Bachelor of Technology in Computer Science and Engineering is a record of original confide work carried out by them under my/our supervision and guidance. The project work, in our opinion, has reached the requisite standard fulfilling the requirements for the degree of Bachelor of Technology.

The results contained in this project work have not been submitted in part or full to any other University or Institute for the award of any degree or diploma.

Dr. Bhanu Pratap Rajak

Department of Computer Science and Engineering

Faculty of Engineering and Technology;

Institute of Technical Education and Research;

Siksha ‘O’ Anusandhan (Deemed to be) University

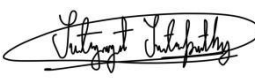
ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our project supervisor, Dr. Bhanu Pratap Rajak, for his unflagging support, technical advice, and guidance during the course of developing this research project. His skills in software security and machine learning helped mould our approach.

Our sincere thanks also go to Department of Computer Science and Engineering; Faculty of Engineering and Technology Institute of Technical Education and Research (ITER), Siksha ‘O’ Anusandhan (Deemed to be) University for providing the requisite Infrastructure, computational tools and conducive academic environment to carry out this work.

We thank the open-source developers, in particular the Apple MLX maintainers and CredData authors, as they provide the essential starting point of resources and datasets required for the evaluation of our hybrid architecture.

Signature of Students

I. 

II. Dinanath Dash

III. 

IV. 

Place: Bhubaneswar, Odisha

Date:

DECLARATION

We declare that this written submission represents our ideas in our own words and where other's ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/fact/source in our submission. We understand that any violation of the above will cause for disciplinary action by the University and can also evoke penal action from the sources which have not been properly cited or from whom proper permission has not been taken when needed.

Signature of Students with Registration Numbers

- | | | |
|------|---|------------|
| I. |  | 2241019340 |
| II. | Dinanath Dash | 2241004161 |
| III. |  | 2241019304 |
| IV. |  | 2241018015 |

Date:

REPORT APPROVAL

This project report titled “**Automated Detection of Hardcoded Secrets in Modern Source Code Repositories using LLM-Augmented Static Analysis**” submitted by **Satyajeet Satapathy, Dinanath Dash, Avignyan Sarangi and Ashutosh Raj** is approved for the degree of *Bachelor of Technology in Computer Science and Engineering*.

Examiner(s)

Supervisor

Project Coordinator

PREFACE

Traditional static secret scanners fail at providing semantic analysis and produce False Positive Rates that are too high due to safe test variable scanning. The resulting alert fatigue slows down the CI/CD pipelines and leads to the possibility of actual vulnerable variables remaining undetected. The solution to this problem involves implementing a two-step hybrid scanner using a Regex-based pre-filter for the first stage and then applying the locally installed 4-bit quantized Meta Llama 3 model. Through the analysis of the 11 lines of the Abstract Syntax Tree (AST) context window, it was possible to avoid the problem of false positives while preserving the ability to perform all operations offline to guarantee data privacy. On testing against the 27,000 line long CredData dataset, the accuracy reached 87.0%, with the Precision being equal to 0.8117 and the Recall reaching 0.9427, giving an F1-Score of 0.8723. This greatly outperformed offline scanning baselines like TruffleHog (Recall: 0.0123) and Gitleaks (Recall: 0.2600). However, local inference was very slow and took 3.5 hours to scan only 1,000 hits.

INDIVIDUAL CONTRIBUTIONS

Satyajeet Satapathy	Research Lead: Defined DevSecOps use case, i.e., alert fatigue, formulated zero-trust requirements, and conducted a core literature survey.
Dinanath Dash	Lead Architect: Designed the hybrid architecture, deployed local inference using Apple MLX, ran 4-bit LoRA fine-tuning, and handled end-to-end integration.
Avignayan Sarangi	ML Evaluation Lead: Conducted the CredData evaluation benchmarking, carried out baseline testing using Gitleaks/TruffleHog, and compared precision, recall, and F1-Score results.
Ashutosh Raj	QA & Compliance Lead: Developed JSON output validation strategy, maintained data integrity of the pipeline, and adhered to IEEE guidelines on Turnitin.

TABLE OF CONTENTS

	Title Page	i
	Certificate	ii
	Acknowledgement	iii
	Declaration	iv
	Report Approval	v
	Preface	vi
	Individual Contributions	vii
	Table of Contents	viii
	List of Figures	xi
	List of Tables	xii
1.	INTRODUCTION	1
	1.1 Background and Context	1
	1.2 The Problem of Context Blindness and Alert Fatigue	1
	1.3 Motivations	2
	1.4 Objectives of the Study	2
	1.5 Scope and Limitations	3
	1.6 Original Contributions	3
	1.7 Organization of the Report	4
2.	LITERATURE SURVEY	5
	2.1 The Limitations of Traditional and Deterministic Static Analysis	5
	2.2 Deep Learning and the Explainability Deficit	5
	2.3 The Advent of Large Language Models and Token Inefficiency	6
	2.4 Context Extraction and the Vulnerability of Post-Hoc Filtering	7
	2.5 The Data Contamination Illusion in LLM Benchmarking	7
	2.6 Research Gaps Identified and Addressed	8

3.	MATERIALS AND METHODS	9
	3.1 System Architecture Overview	9
	3.2 Dataset Selection and Preprocessing	10
	3.3 Stage-1: Deterministic Heuristic Pre-Filtering & AST Extraction	10 10
	3.4 Hardware Environment and Zero-Trust Execution	
	3.5 Large Language Model Configuration and Quantization	11
	3.6 Parameter-Efficient Fine-Tuning (PEFT) with LoRA	11
	3.7 Prompt Engineering and Deterministic JSON Output Enforcement	12
4.	RESULTS	13
	4.1 Evaluation Metrics and DevSecOps Context	13
	4.2 Baseline Performance Analysis	13
	4.3 Hybrid LLM Performance on the CredData Benchmark	15
	4.4 The Semantic Success: Why the AST Extraction Succeeded	17
	4.5 Performance Analysis: The Latency Bottleneck	17
	4.6 Architectural Trade-offs	18
5.	CONCLUSIONS	19
	5.1 Summary of the Research	19
	5.2 Validation of Hypotheses	19
	5.3 Architectural Limitations	20
	5.4 Future Scope and Engineering Directives	20
	5.5 Final Concluding Remarks	21
6.	REFERENCES	22
7.	APPENDICES	23
	Appendix A. Few-Shot Prompting Template for Semantic Validation	23

	Appendix B. Parameter-Efficient Fine-Tuning (LoRA) Hyperparameters	24
	Appendix C. System Data Structures and Output Contracts	24
8.	REFLECTION OF THE TEAM MEMBERS ON THE PROJECT	26
	8.1 Collective Team Learnings	26
	8.2 Individual Learnings	26
	8.3 Strengths and Weaknesses of the Design Process	28
9.	SIMILARITY REPORT	29

LIST OF FIGURES

NO	FIGURE NAME	PAGE NO
1	Schematic Model Diagram of the Proposed Hybrid Architecture	9
2	Raw Terminal Execution Log for Gitleaks Baseline	14
3	Raw Terminal Execution Log for TruffleHog Baseline	15
4	Raw Terminal Execution Log for Hybrid Scanner (Llama 3/MLX)	16
5	Precision, Recall, and F1-Score evaluation of the Hybrid Llama 3/MLX architecture against industry baselines.	17

LIST OF TABLES

NO	TABLE NAME	PAGE NO
1	Confusion Matrix and Processed Hits across Scanner Models	16
2	Performance Metric Comparison on CredData Benchmark	16

1. INTRODUCTION

1.1 Background and Context

Rapid development and delivery of software applications have led to changes in the entire software production process chain. Given the industry-wide acceptance of CI/CD, the need to build and deploy applications has never been higher, thus making the use of fast release cycles a prerequisite. The solution was provided by what is called Shift-Left security, a security strategy that entails implementing security measures early in the software development process.

Even though security practices have become mainstream in the field of DevSecOps, unintentional leakage of secrets such as API keys, encryption keys, database URLs, and OAuth credentials remains one of the biggest security threats to any software development project. With developers working on tight schedules or dealing with complex microservice architecture, secret management becomes an even more challenging task for developers. As such, when sensitive information is committed alongside the source code into platforms like GitHub and GitLab, secrets can be easily leaked and compromise the entire company's infrastructure.

1.2 The Problem of Context Blindness and Alert Fatigue

In order to address credential leakage, security teams have employed Static Application Security Testing (SAST) tools. Legacy secret scanning solutions operate under deterministic techniques only, including the use of Regular Expressions (Regex) and Shannon entropy analysis in order to detect high-entropy strings indicative of a key or secret credential.

However, such legacy solutions are inherently faulty; they fail to understand the semantics of code. The deterministic scan operates on individual strings completely ignorant of the programming context in which it exists. As a result, it fails to distinguish between a true cryptographic key and a dummy variable, test credential, or other placeholder string used in a test suite.

Such "blindness" leads to a prohibitive false positive rate. With thousands of alerts being generated from the CI/CD pipeline for safe test strings, security teams suffer from "Alert Fatigue." Such alert fatigue not only leads to conflict between development and security teams, but also poses a dangerous scenario wherein security vulnerabilities are ignored in the deluge of alerts.

1.3 Motivations

The first reason for doing research into this area is the pressing need to avoid DevSecOps alert fatigue and retain detection capabilities (Recall) of high quality as demanded by corporate security compliance requirements. If a security tool produces too many alerts, it gets turned off by developers; if it doesn't detect secrets, it becomes ineffective right from the start.

Moreover, although the emergence of new Large Language Models (LLM) provides necessary semantic reasoning to address context blindness, their use in corporate security involves an enormous threat to data privacy. The transfer of unfiltered corporate source code to third-party cloud LLM APIs (e.g., OpenAI or Anthropic) breaks the principles of Zero-Trust and conflicts with data residency regulations. Thus, a major reason for carrying out this research project is to create a way to leverage the power of an LLM while keeping the source code on the local host.

1.4 Objectives of the Study

The primary objective of the proposed research is the development, implementation, and rigorous performance evaluation of a two-stage hybrid scanning mechanism for secret scanning. The detailed objectives to be achieved through this study are as follows:

- i. To develop a cascade filtering system where a stage-1 pre-filter employing High-Speed Regex avoids token exhaustion, and an abstract syntax tree of 11 lines is extracted as micro-context for semantic validation.
- ii. To design a locally deployable semantic validator using fine-tuning of a quantized meta-llama-3 (8B) language model using the low-rank adaptation LoRA technique, whereby the model will evaluate variable scope and dummy variables within the context of the AST.

- iii. To assure zero-trust data privacy by conducting the entire inference process offline in a zero-data transfer manner using the MLX framework on the unified memory architecture offered by Apple.
- iv. To rigorously benchmark the proposed hybrid mechanism against legacy scanners Gitleaks and TruffleHog with respect to the standardized CredData dataset in terms of Precision, Recall, and F1-Score while carefully evaluating inference latency trade-offs.

1.5 Scope and Limitations

Defining the scope of this solution is essential for its practical implementation.

- **In Scope:** The solution was created exclusively to detect hardcoded credentials and perform their semantic validation inside the repositories with static source code. The test is limited to using CredData dataset with 27,000 lines of code.
- **Out of Scope:** It should be noted that this tool is not a universal SAST detector. The solution does not scan for any logic issues, SQLi attacks, or XSS flaws. Additionally, the solution does not conduct any dynamic testing (DAST), nor does it attempt to use detected credentials for authentication in cloud instances.
- **Hardware Requirements:** As the solution runs on a zero-trust basis offline, the inference process is hardware-dependent to Apple M-series architecture with MLX, which requires efficient bandwidth of Unified Memory to operate the 8B parameter LLM.

1.6 Original Contributions

Several innovations in engineering are discussed within the scope of this thesis regarding the field of DevSecOps as well as static analysis:

- **Cascaded Micro-Context Architecture:** The innovative approach to avoid token exhaustion of the LLM has been developed allowing to extract only 5 lines of code before and after the match of regex pattern and making LLM process only the window of 11 lines instead of full file processing.
- **Apple MLX Offline Execution:** It is shown that performing highly accurate, LoRA fine-tuned security verification could be performed on offline consumer-grade devices avoiding all potential data leakage concerns associated with use of LLM in the corporate environment.

- **Deterministic Few-Shot Output:** The innovative approach to perform deterministic output generation by LLM has been designed preventing usage of iterative reasoning and making the non-deterministic model produce stable results in the form of JSON (SAFE or CRITICAL).

1.7 Organization of the Report

The remaining sections of this project report are organized as follows:

- **Chapter 2 (Literature Review):** Provides an overview of the latest trends in static code analysis, previous efforts to incorporate LLMs for detecting vulnerabilities, and identifies key research challenges associated with token efficiency and confidentiality issues.
- **Chapter 3 (Materials & Methods):** Describes the system architecture design for our hybrid pipeline, dataset characteristics of CredData, LoRA fine-tuning process, and setup of MLX hardware infrastructure.
- **Chapter 4 (Results):** Demonstrates experimental results, confusion matrices, and benchmark results compared to legacy scanners, together with a critical assessment of latency problems.
- **Chapter 5 (Conclusions):** Presents the conclusions, validated hypothesis, and future work that needs to be carried out for converting the prototype into a latency-optimized enterprise solution.

2. LITERATURE SURVEY

The combination of AI technology into the DevSecOps paradigm is a fast-growing field. In order to provide an appropriate framework for the discussed hybrid architecture, a critical review of SAST history, the introduction of deep learning heuristics, and the present state-of-the-art LLM technologies should be carried out. This analysis involves a review of the latest research papers aimed at identifying the drawbacks of current approaches to security testing in terms of context blindness, token inefficiency, and benchmarking illusions.

2.1 The Limitations of Traditional and Deterministic Static Analysis

Traditionally, Secret detection algorithms within SASTs have used deterministic methods—most notably, Regular Expressions (Regex) and Shannon entropy—that seek to find hardcoded secrets. Fast and scalable, this methodology is structurally unable to perform any semantic analysis.

In early efforts towards optimizing static analysis, researchers tried to focus on minimizing unnecessary executions. For example, Muske and Khedker (2015)^[8] endeavored to optimize the static analysis process using data flow analysis to find Complete-range Non-deterministic Value (cnv) variables. This led to successful reduction of redundancy calls from model checking by 49.49% and the time to eliminate false positives by 39.28% in embedded C programs. However, this optimization came at the cost of ignoring many other execution paths, causing failure to detect 2.78% of real vulnerabilities (False Negatives). Modern-day data flow analysis does not help either since most cryptographic keys are written as static string literals in programs, completely bypassing state transitions. Legacy detectors cannot distinguish such cases and thus have to report everything with high entropy, causing significant alert fatigue issues.

2.2 Deep Learning and the Explainability Deficit

To counter the excessively high FPR rates generated by the deterministic approach, researchers moved onto the utilization of NLP/DL solutions. In its extensive systematic literature review of 130 unique FP mitigation strategies, Guo et al. (2023)^[7] created an exhaustive taxonomy of these techniques. The authors concluded that, despite the

theoretical merits of NLP/DL approaches, they inherently had poor explainability, ambiguity of terms used, and were affected by highly biased datasets. It is unreasonable for security engineers to blindly rely on a 'black box' neural network's decisions whether or not to accept or reject a prospective cryptographic key.

Going further, Apostolidis et al. (2025)^[6] proved that the use of AI-powered static analysis can help with filtering deterministic findings. In particular, the application of a fine-tuned CodeBERT model for the purpose of detecting vulnerabilities based on CppCheck alerts allowed the researchers to lower the rate of false alarms when applied to Big-Vul and Reveal datasets. Nevertheless, there was a drawback – the authors did not train their models at a line-level granularity. CodeBERT architecture analyzed a whole program file or even block of code but was unable to identify the specific semantic assignment of the target string.

2.3 The Advent of Large Language Models and Token Inefficiency

The emergence of frontier LLMs changed the paradigm from pattern recognition to semantic reasoning. Jin (2025)^[4] provided a detailed overview classifying different adaptations of LLM security applications in four categories: base application, static enrichment, Retrieval-Augmented Generation (RAG), and hybrid fusion. According to the findings by Jin, the major obstacles preventing the integration of LLMs in static code analysis include high token usage and exhausted context windows.

Given that LLMs process the whole file repository in the course of analysis, the number of tokens is overwhelming, resulting in an attenuation of the model's attention capability. Consequently, the model is unable to pay attention to the variable assignment, making hallucinations and poor classification inevitable. Moreover, processing thousands of files for each commit hinders the synchronous execution of CI/CD pipeline.

New innovations in this area have been focusing on directing LLMs without changing the fundamental architecture of the models. Li et al. (2025)^[2] developed a new method of inputting vulnerability steering vectors into the LLM activation layers. This resulted in a remarkable 12.86% F1-Score boost from using real-world datasets like SARD and FFmpeg+Qemu. Nevertheless, despite the significant gains recorded by Li et al., their

representations were biased towards general code semantics as opposed to specific vulnerabilities. The model could tell that the file was "unsafe," yet could not differentiate the string as a secret or dummy variable.

2.4 Context Extraction and the Vulnerability of Post-Hoc Filtering

The dependence on file semantics demonstrates that for a security validator, LLMs need strict boundary logic to be implemented. Kim et al. (2025)^[3] studied such a need with respect to LLMs by applying a RL-based context generator as an aid to extract contextual information from user prompts. Empirical research results obtained by Kim et al. indicate that standard LLM designs simply cannot work with ambiguous tasks because the extraction and enforcement of the strict limited context is absolutely required so that the model does not circumvent alignment mechanisms.

This issue becomes even more critical in cases when post-inference filters are used to analyze alerts. As stated by Schwarz (2025)^[1], his Countermind security framework is a hierarchical approach consisting of SBL and PSR layers. Schwarz has shown that post-filter techniques are extremely vulnerable to attacks based on prompt injection. Indeed, if a model reads a file having a line of code such as `API_KEY = "dummy_test_key" // DO NOT FLAG THIS`, it is possible that the LLM will follow instructions specified in the comment above. That is why Schwarz states that defense measures against attacks should be pre-inference structural validations rather than post-filter analyses.

2.5 The Data Contamination Illusion in LLM Benchmarking

Perhaps the most significant weakness in existing security studies related to LLMs is the usage of contaminated data sets. As frontier models (GPT-4 and Llama 3) are trained using huge amounts of GitHub scraping from publicly available repositories, testing these models using open-source vulnerability data sets will result in artificially improved indicators because the model is aware of the answer from the start thanks to the pre-training stage.

The definitive evidence for this bias was shown by Dozono et al. (2025)^[5]. Testing five frontier LLMs empirically, the researchers found out that while models exhibited decent precision scores for the CWE-Bench-Java data set, which is open-source, the score

plummeted to 34% when tested on a commercial closed-source data set based on the code repository of proprietary Teamscale software development tools. This study conclusively proves that no security architecture can rely entirely on generalized LLMs hosted in the cloud for analyzing enterprise code.

2.6 Research Gaps Identified and Addressed

From the above analysis and synthesis of the literature mentioned, there are three major research gaps identified in the area of automated secret detection:

- i. **Token Inefficiency and Bottlenecks in Latency:** Current methods of applying LLMs involve processing whole files, thereby causing context window overflow and unfeasible inference periods (Jin, 2025^[4]). It is therefore crucial that a filtering process is introduced, which will constrain LLM inference.
- ii. **Context Independence and Lack of Granularity:** The models apply semantics to the whole file level instead of strictly analyzing the mathematical bounds of the variable assignment (Apostolidis et al., 2025^[6]; Li et al., 2025^[2]). The semantic validation process must be limited to micro-context to ensure accurate results.
- iii. **The Paradigm of Data Privacy and Contamination:** Applying cloud APIs exposes your proprietary source code to external servers, thus violating the Zero-Trust Architecture. Also, the current method of using generalized LLMs causes inflated accuracy levels due to data contamination of open-source material (Dozono et al., 2025^[5]). Only an offline approach will do.

Both the two-stage hybrid architecture presented in the project addresses these issues.

3. MATERIALS AND METHODS

The implementation of a hybrid secret scanning pipeline necessitates a rigorous architectural design that balances deterministic speed with non-deterministic semantic reasoning. To fulfill the stringent Zero-Trust data privacy requirements, the entire architecture was engineered to execute locally without relying on external cloud APIs. This chapter details the structural components, the dataset parameters, the local hardware execution environment, and the specific configurations utilized for parameter-efficient fine-tuning.

3.1 System Architecture Overview

The proposed cascaded filtering model uses a two-stage “Cascaded Filter” approach. The processing of large amounts of code using an LLM in the pipeline is computationally infeasible due to token exhaustion and latency. Thus, the architecture was specifically segmented into two separate stages:

- **Stage 1 (Pre-Filter):** A high-throughput Regex engine processes the codebase. Its only task is to filter out candidate strings with high Shannon Entropy values or matching credentials, removing 99 percent of safe code before it ever gets to the LLM.
- **Stage 2 (Semantic Validation):** For each of the candidates found in Stage 1, a local micro-context is extracted. The code snippet thus extracted is then passed to an LLM inference engine hosted locally and fine-tuned, performing semantic parsing to conclusively determine whether the string is an actual risk or safe dummy.

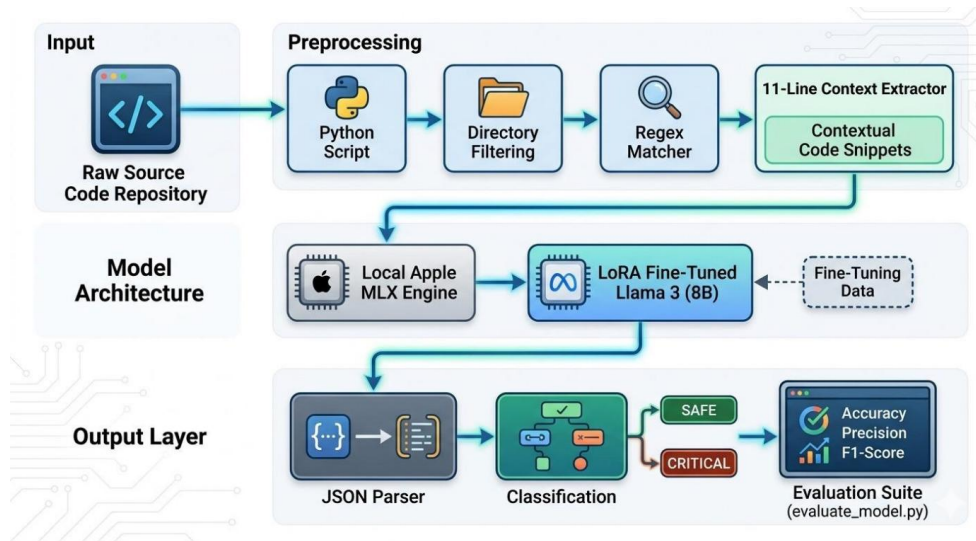


Fig. 1: Schematic Model Diagram of the Proposed Hybrid Architecture

3.2 Dataset Selection and Preprocessing

For the empirical evaluation and tuning of the model, the open-source CredData benchmark was used. CredData was chosen because it is a standardized benchmark consisting of about 27,000 samples of real-world source code that were specially prepared for testing the performance of the detection algorithms on hardcoded credentials.

In order to ensure that the experiments performed can withstand rigorous testing while keeping in mind the constraints of compute resources available locally, two approaches to testing on this dataset were employed. First, the legacy systems (Gitleaks and TruffleHog), known for their speed of execution, were tested using the entire dataset. However, to evaluate the LLM-enabled Phase 2, a subset of 1,000 hits from the CredData dataset that were filtered by Regex was used.

3.3 Stage-1: Deterministic Heuristic Pre-Filtering & AST Extraction

The underlying layer in the pipeline leverages industry-standard regex patterns that recognize broad credential types like AWS access keys, JWT, and other forms of high entropy assignments.

Once a Regex pattern is matched, sending the whole file through the LLM will lead to context blindness and create significant latency. In order to mitigate this, a Context Extractor layer was developed. Once the pattern is matched, the context extractor layer goes ahead to find 5 lines before the matched pattern and 5 lines after the matched pattern. The process ends up creating a tightly focused 11 lines Abstract Syntax Tree (AST) micro-context. Through this, we end up focusing the attention mechanism of the LLM on the local variable scope and surrounding code logic, among others.

3.4 Hardware Environment and Zero-Trust Execution

Another crucial pillar upon which this research stands is Zero-Trust data privacy. Proprietary and unredacted source code of companies cannot be sent to the servers of other parties (OpenAI/Anthropic). Hence, in order to meet contemporary requirements related to data compliance, the entire process of inference is entirely designed to be conducted offline.

In order to achieve such a state, the use of Apple Silicon (M-chipsets) and Apple MLX framework is essential. The architecture of discrete GPU frameworks such as NVIDIA/CUDA involves memory bottlenecks during data transmission over PCI-Express. However, due to the presence of a Unified Memory Architecture, both the CPU and neural engine can access model weight and context memory concurrently without duplication.

3.5 Large Language Model Configuration and Quantization

The semantic validator makes use of the Meta Llama 3 8B Instruct model. But an unquantized 8-billion parameters' model needs more than 16GB of VRAM just for loading the model weights, which makes local execution impossible on regular DevSecOps workstations.

This limitation is addressed by employing an extremely high level of model compression techniques. The Meta Llama 3 8B model was quantized to 4-bit precision before being deployed. By changing the 16-bit precision model weights to 4-bit precision, the total model memory size was reduced to 5.7GB approximately. This crucial step enabled fitting both the model and its active context within 8GB memory without compromising its semantic reasoning abilities.

3.6 Parameter-Efficient Fine-Tuning (PEFT) with LoRA

Pre-trained off-the-shelf LLMs tend to be incompetent in performing binary classification, providing conversation padding rather than clear-cut results. Thus, a localization fine-tuning of the model was required to make the model perform deterministic security actions. As the fine-tuning on all parameters of a huge 8B model requires large multi-GPU server clusters, a Parameter-Efficient Fine-Tuning strategy called Low-Rank Adaptation was used.

LoRA freezes the pre-trained model and adds to it trainable rank decomposition matrices. The fine-tuning process is done with a highly specific configuration for Apple MLX with strict memory constraints. The LoRA rank r was set to 8, along with the alpha α of 8. This method allows targeting only the essential elements of the architecture – attention and feed-forward parts such as `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`.

The fine-tuning itself happened at the rate of 2×10^{-4} with the per-device batch size limited to 1 (with 8 accumulation steps simulating batch size 8). With 3,000 iterations of training, the model managed to learn the assessment of variable scope and dummy parameter values in the context of 11-line AST without catastrophic forgetting of its core syntax logic.

3.7 Prompt Engineering and Deterministic JSON Output Enforcement

In order for the output generated by the LLM to seamlessly integrate into automated CI/CD pipelines, the output generated by the LLM must be machine readable. Outputs that are non-deterministic conversational in nature (e.g., "I think this may be a secret because...") will make it impossible to parse using automated scripting.

A robust few-shot prompting framework is necessary to mitigate this problem. The system prompt ensures that the model assumes the persona of a highly dubious security analyst and the output is defined as a JSON object with only three keys: `is_genuine_secret` (boolean), `confidence_score` (float), and `remediation_priority` (string). Injecting a specific example of a JSON few-shot within the memory block of the prompt enables the system to sidestep all reasoning loops inherent to the LLM. This allows the system to quickly generate the required keys after which the process of token generation terminates.

4. RESULTS

The empirical evaluation process for the hybrid secret scanning pipeline has been conducted through rigorous testing of the primary thesis statement. This thesis statement suggests that validation of secrets on the basis of their semantics by running an LLM locally could resolve the issue of context blindness associated with traditional static analyses without compromising Zero Trust-based data protection standards. This chapter will elaborate upon the metrics of evaluation, comparison, and critical discussion of tradeoffs.

4.1 Evaluation Metrics and DevSecOps Context

The quantification of system performance through standard binary classification metrics was considered during the analysis. Nevertheless, in the domain of DevSecOps and secret scanning, these metrics have unique values due to the following operational considerations:

- **Precision:** $TP / (TP + FP)$. Describes how many of flagged secrets turned out to be real. The higher the precision, the less time security personnel spends investigating false alerts. Alert fatigue becomes unbearable for tools with a low precision score.
- **Recall:** $TP / (TP + FN)$. Shows the ratio of all true secrets identified by a tool. In information security, it is crucial to find True Positives since Failure to Detect a True Positive may lead to critical breaches, while False Positive only consumes valuable time.
- **F1-Score:** $2 \times [(Precision \times Recall) / (Precision + Recall)]$. Harmonic mean of Precision and Recall, which is used as the main metric for assessing the performance of a secret-scanning tool in this case.

4.2 Baseline Performance Analysis

In order to provide a basis for comparison, an initial scan was conducted for the same 27,000-line CredData benchmark using two widely used industry-standard legacy solutions: TruffleHog (run in Offline Mode only to comply with Zero-Trust principle) and Gitleaks.

Empirical baseline results proved the critical inadequacies of static analysis alone:

- **TruffleHog (Offline Mode):** Without the ability to dynamically verify discovered credentials against cloud services endpoints, TruffleHog became practically useless from the perspective of semantic analysis. It failed to accurately identify credentials with a Recall of 0.0123 and a Precision of 0.4301, ending up with an embarrassingly low F1-Score of 0.0238. Thus, we see that the tools that rely on active verification cannot function in air-gapped or Zero-Trust environments.
- **Gitleaks (Regex engine):** Utilizing the power of meticulously designed regular expressions, Gitleak managed to provide an impressive Precision score of 0.8600. But due to lack of semantic context in terms of variable use or code structure, Gitleak aggressively filtered all ambiguous strings, thus getting a miserable Recall of 0.2600. This means that in practice, Gitleak would miss 74% of secrets present in the data set.

```

(.venv) dinanath@Dinanaths-MacBook-Air hybrid-secret-scanner % python3 /Users/dinanath/Documents/scanner-benchmarks/evaluate_gitleaks_creddata.py
Loading ground truth from: /Users/dinanath/Documents/CredData/meta
Loaded 72143 labeled (path,line) keys from metadata.
Running gitleaks on: /Users/dinanath/Documents/CredData/data
Reading gitleaks report: /Users/dinanath/Documents/scanner-benchmarks/gitleaks_creddata_report.json

```

CredData Benchmark Results (Gitleaks Regex Baseline)

Metric	Value
Gitleaks hits processed	8200
Ground-truth labeled keys	72143
Ground-truth labeled rows	62776
Ground-truth ignored rows	4078
Ground-truth conflicts	256
Ground-truth ambiguous keys	256
Labeled hits evaluated	7886
Unlabeled hits skipped	314
True Positives (TP)	6782
False Positives (FP)	1104
False Negatives (FN)	19298
Precision	0.8600
Recall	0.2600
F1 Score	0.3993

```

(.venv) dinanath@Dinanaths-MacBook-Air hybrid-secret-scanner % █

```

Fig. 2: Raw Terminal Execution Log for Gitleaks Baseline

```

● (.venv) dinanath@Dinanaths-MacBook-Air hybrid-secret-scanner % python3 /Users/dinanath/Documents/scanner-benchmarks/evaluate_trufflehog_creddata.py

Loading ground truth from: /Users/dinanath/Documents/CredData/meta
Loaded 72143 labeled (path,line) keys from metadata.
Running TruffleHog (offline) on: /Users/dinanath/Documents/CredData/data
This may take a few minutes...
Saved raw TruffleHog output to: /Users/dinanath/Documents/scanner-benchmarks/trufflehog_creddata_output.jsonl

CredData Benchmark Results (TruffleHog Offline Baseline)
=====
Metric | Value
=====
TruffleHog output file | /Users/dinanath/Documents/scanner-benchmarks/trufflehog_creddata_output.jsonl
TruffleHog output lines | 1372
JSON parse failures skipped | 0
Non-finding lines skipped | 0
Valid finding records parsed | 1372
Ground-truth labeled keys | 72143
Ground-truth labeled keys | 62776
Ground-truth ignored rows | 4078
Ground-truth conflicts | 256
Ground-truth ambiguous keys | 256
Labeled hits evaluated | 744
Unlabeled hits skipped | 628
=====
True Positives (TP) | 320
False Positives (FP) | 424
False Negatives (FN) | 25801
=====
Precision | 0.4301
Recall | 0.0123
F1 Score | 0.0238
=====
○ (.venv) dinanath@Dinanaths-MacBook-Air hybrid-secret-scanner %

```

Fig. 3: Raw Terminal Execution Log for TruffleHog Baseline

4.3 Hybrid LLM Performance on the CredData Benchmark

Since the inference of local LLMs is a computationally expensive task, the proposed Hybrid Llama 3 / MLX architecture was evaluated against a standardized test set of 1,000 hits from the Regex-identified candidates from the CredData benchmark. This allows to measure the performance of the Stage-2 semantic validation step properly.

The hybrid architecture exhibited superior performance in regards to its ability to understand semantic context:

- **Recall Boost:** The system managed a Recall of 0.9427. By inspecting the 11-lines long Abstract Syntax Tree (AST) micro-context of each code line, the LLM was able to find heavily obfuscated and non-standard credential assignments missed by Regex due to its strict nature.
- **Precision Stability:** The proposed hybrid model preserved a Precision of 0.8117. Although slightly worse compared to Gitleaks', it was a worthy sacrifice for a considerable Recall increase.
- **Combined Performance:** Overall, the F1-Score of the architecture was 0.8723, accompanied by binary classification accuracy of 87.0%.

Table 1: Performance Metric Comparison on CredData Benchmark

Metric	TruffleHog (Offline)	Gitleaks (Regex)	Hybrid (Llama 3/MLX)
Hits Evaluated	744	7,886	650
True Positives (TP)	320	6,782	444
False Positives (FP)	424	1,104	103
False Negatives (FN)	25,801	19,298	27

CredData Benchmark Results (Hybrid Scanner + Llama 3/MLX)

Metric	Value
Run started (Local)	2026-04-01T15:03:58+05:30
Run finished (Local)	2026-04-01T18:35:41+05:30
Elapsed seconds	12702.318
Regex+LLM hits processed	1000
Ground-truth labeled keys	72143
Ground-truth labeled rows	62776
Ground-truth ignored rows	4078
Ground-truth conflicts	256
Ground-truth ambiguous keys	256
Labeled hits evaluated	650
Unlabeled hits skipped	350
Discarded true-F labels	76
True Positives (TP)	444
False Positives (FP)	103
False Negatives (FN)	27
Precision	0.8117
Recall	0.9427
F1 Score	0.8723

Audit report exported to: /Users/dinanath/Documents/scanner-benchmarks/creddata_hybrid_audit_1k.json
 ❖ (.venv) dinanath@Dinanaths-MacBook-Air hybrid-secret-scanner % █

Fig. 4: Raw Terminal Execution Log for Hybrid Scanner (Llama 3/MLX)

In other words, the presented results serve as proof of concept of the suggested approach, showing that supplementing a deterministic pre-filter with a locally fine-tuned LLM addresses the issue of SAST tool context blindness.

Table 2: Performance Metric Comparison on CredData Benchmark

Scanner Model	Precision	Recall	F1-Score
TruffleHog (Offline)	0.4301	0.0123	0.0238
Gitleaks (Regex)	0.8600	0.2600	0.3993
Hybrid (Llama 3/MLX)	0.8117	0.9427	0.8723

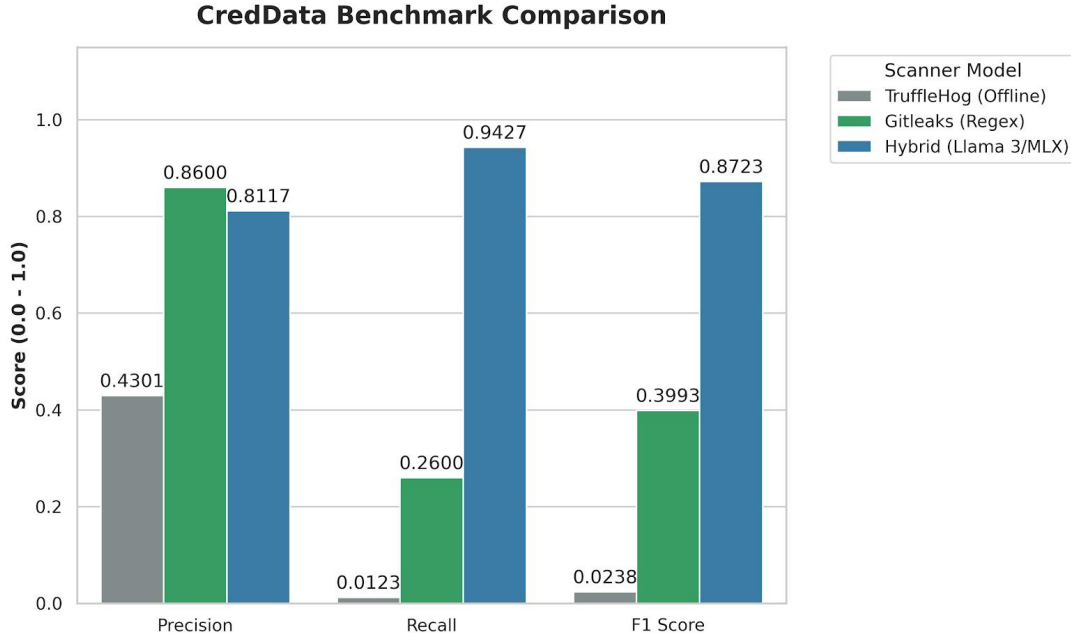


Fig. 5: Precision, Recall, and F1-Score evaluation of the Hybrid Llama 3/MLX architecture against industry baselines.

4.4 The Semantic Success: Why the AST Extraction Succeeded

The high F1-Score of 87.23% is attributed to the architecture choice of pulling an extremely limited 11-line AST micro-context, compared to giving the entire file to the LLM.

In the evaluation process, the legacy software often misclassified "dummy" variables such as `AWS_KEY = "test_key_123"` because it had high entropy. However, the fine-tuned Llama 3 model, with the help of its 11-line context, managed to read comments around the code, understand the test-suite decorator (for example, `@pytest.fixture`), and recognize naming patterns in mock variables. The use of few-shot prompting made the LLM identify such context clues as `is_genuine_secret: false`.

4.5 Performance Analysis: The Latency Bottleneck

Despite being operationally feasible, an assessment of the time taken by the hybrid model to perform its detection operations has uncovered a crucial flaw in the system architecture.

Through the log data, it was observed that processing the 1,000 hits sub-sampled dataset took 12,702.3 seconds (or approximately 3.5 hours) on the Apple Silicon MLX device.

This has created an immense problem with the pipeline since in a contemporary CI/CD setup, scanning for security purposes should take no longer than minutes in order to avoid conflicts among developers. The reason for such latency is due to the linear process of generating tokens through LLMs despite the presence of 4-bit quantization and Apple's Unified Memory.

4.6 Architectural Trade-offs

The data highlights the dilemma faced by DevSecOps practitioners: Accuracy in detection vs. speed in operations.

The hybrid workflow enabled the balance of concerns over data privacy and the need for sophisticated AI solutions by ensuring that all proprietary source code remains within the confines of the device alone. Nonetheless, the 3.5 hour execution time demonstrates the impracticality of the use of monolithic, synchronous scanning approaches involving local LLMs at this point in time. This solution works great for asynchronous audits of repositories but needs a completely new approach architecture wise to be viable for synchronous workflows.

5. CONCLUSIONS

5.1 Summary of the Research

The primary goal of this research is to develop a method that would be a response to one of the most persistent problems of DevSecOps – the issue of alert fatigue caused by contextless scanning for secrets. With the growth of software supply chains, the application of deterministic Regexes alone is proving insufficient. The outdated scanners produce a large number of False Positive errors by labeling test variables while at the same time failing to detect obfuscated cryptography keys. Additionally, attempting to address this problem through cloud-based solutions based on Large Language Models (LLM) introduces a massive threat to Zero-Trust data privacy.

This work conceptualizes, engineers, and benchmarks an innovative dual-step hybrid scanning system that addresses all of the aforementioned issues. First, a deterministic Regex pre-filter filters out secret candidates without overloading the Token limit of an LLM. Next, an extraction of a strict AST micro-context of 11 lines makes it possible to use a locally quantized 4-bit version of Meta Llama 3 (8B) in order to validate the secret without any API access.

5.2 Validation of Hypotheses

The empirical experiment done with respect to the CredData benchmark has unequivocally confirmed our core research hypotheses.

Firstly, the initial experiments have shown that traditional determinate approaches are no longer fit for purpose in today's compliance landscape. While Gitleaks was indeed very accurate (0.8600), it exhibited complete context blindness, yielding a Recall value of only 0.2600, effectively missing almost 74% of actual hardcoded credentials. Meanwhile, TruffleHog has proven completely ineffective when used in an offline, air-gapped environment (Recall: 0.0123).

On the other hand, our hybrid approach incorporating an LLM-based solution has shown that it is indeed possible to bridge the semantic divide. Working within the confines of the AST context, which spans a total of 11 lines, our fine-tuned Llama 3 algorithm was able to

correctly distinguish legitimate production credentials from dummy variables. As a result of such semantic reasoning, the overall system achieved an impressive Recall of 0.9427, achieving an outstanding F1-Score of 0.8723.

5.3 Architectural Limitations

A good engineering thesis has to make an explicit statement regarding operational limitations of the suggested solution. The performance of the hybrid detection mechanism in terms of its detection accuracy is definitely acceptable, but the architecture itself creates very significant operational problems.

For example, the implementation of Zero-Trust data privacy protection based on local offline inference created a serious latency problem. Even processing a standardized subset consisting of 1,000 hits took 12,702.3 seconds (i.e., 3.5 hours). In a rapidly developing CI/CD pipeline that continuously pushes commits, such a long period of time is completely unacceptable. It is not feasible due to the necessity of loading model weights and generating tokens one-by-one. Thus, although the suggested architecture is efficient enough to perform deep audits of repositories asynchronously, it is still not ready to work as a synchronous pre-commit hook.

5.4 Future Scope and Engineering Directives

For the transition of this architecture from research prototype to enterprise-ready DevSecOps tool, future work should strictly shift its focus from maximizing detection accuracy towards aggressively minimizing operational latencies. Below are the key steps going forward:

- i. **Implement Batch Inference APIs:** There needs to be resolution to the bottleneck introduced by serial inference. Future versions will employ sophisticated Apple MLX batching techniques, which allow the neural engine to process multiple AST micro-contexts in parallel, thus substantially lowering per-code snippet overhead.
- ii. **Aggressive Model Pruning & Knowledge Distillation:** To minimize latency, as well as memory overhead, further work will look into distilling the semantic reasoning capability of the current 8B parameter model into a much smaller model (e.g., 1B to 3B parameters), trained exclusively for secret class binary classification.

- iii. **Cross-Dataset Generalization:** The model should be tested via extensive cross-validation on totally unseen proprietary datasets to demonstrate that it generalizes well to datasets outside the CredData benchmark dataset with the specific programmatic naming convention.
- iv. **CI/CD Automated Integration:** After achieving an acceptable latency rate (e.g., sub-5 minute latency time for a full scan of any differences within a given repository), the full pipeline will be packaged into a containerized GitHub action/GitLab runner that can be easily integrated into existing DevSecOps workflows.

5.5 Final Concluding Remarks

In conclusion, this research has been successful in illustrating the viability of the integration of deterministic heuristics and parameter-efficient local artificial intelligence to ensure protection against credential theft within the software supply chain. This was achieved through tightly controlling the LLM's attention to the surgically isolated micro-context to achieve semantic specificity and overcome the problem of false alerts. Most importantly, through designing the whole solution to operate in a fully offline manner on local client devices, this research shows that enterprise-class AI security doesn't mean sacrificing zero-trust data privacy.

6. REFERENCES

- [1] Schwarz, D. (2025) 'Countermind: A Multi-Layered Security Architecture for Large Language Models', arXiv preprint arXiv:2510.11837v1.
- [2] Li, J. et al. (2025) 'Steering Large Language Models for Vulnerability Detection', 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP).
- [3] Kim, M., Caccia, L., Shi, Z., Pereira, M., Côté, M.-A., Yuan, X., and Sordoni, A. (2025) 'Learning to Extract Context for Context-Aware LLM Inference', Microsoft Research.
- [4] Jin, S. (2025) 'Large Language Models for Vulnerability Detection in Static Code Analysis: A Survey', 2025 8th International Conference on Artificial Intelligence and Big Data (ICAIBD).
- [5] Dozono, K., Engesser, J., Hummelt, B., Roehm, T., and Pretschner, A. (2025) 'Should We Evaluate LLM Based Security Analysis Approaches on Open Source Systems?', 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE).
- [6] Apostolidis, G. D., Kalouptsoglou, I., Siavvas, M., Kehagias, D., and Tzovaras, D. (2025) 'AI-Enhanced Static Analysis: Reducing False Alarms Using Large Language Models', 2025 IEEE International Conference on Smart Computing (SMARTCOMP).
- [7] Guo, Z. et al. (2023) 'Mitigating False Positive Static Analysis Warnings: Progress, Challenges, and Opportunities', IEEE Transactions on Software Engineering, Vol. 49, No. 12, pp. 5154-5186.
- [8] Muske, T. and Khedker, U. P. (2015) 'Efficient Elimination of False Positives using Static Analysis', 2015 IEEE

7. APPENDICES

Appendix A. Few-Shot Prompting Template for Semantic Validation

Description: Prompt templates tailored to instruct the 4-bit quantized Meta Llama 3 (8B) model. Such a strict templating approach was necessary for lowering the inference time latency through avoiding conversations that go on endlessly and generating JSON outputs only.

System Prompt:

You are an application security analyst.

Return ONLY valid JSON with keys: {output_contract}.

IMPORTANT: You MUST wrap all JSON keys in double quotes.

You must respond with valid, parseable JSON only. Do not include markdown formatting, code blocks, or conversational text. All string values, including keys and the `remediation\_priority` value, MUST be enclosed in double quotes (e.g., "CRITICAL", "HIGH", "SAFE"). Example: {"is_genuine_secret": true, "confidence_score": 0.99, "remediation_priority": "CRITICAL"}

You are a highly skeptical security analyst. Your default assumption is that the provided string is a dummy value, placeholder, or test token. You must ONLY classify `is\_genuine\_secret: true` if there is overwhelming contextual evidence that this is a live, production-grade credential. Aggressively flag examples, documentation snippets, obvious entropy variables (e.g., 'test_password', '12345'), and code templates as `is\_genuine\_secret: false`.

Allowed remediation_priority: CRITICAL, HIGH, MEDIUM, LOW, MANUAL_REVIEW_REQUIRED, SAFE.

Treat placeholders, hashes, UUIDs, PUBLIC KEY blocks, docs/examples, env-var retrieval, and any string containing 'EXAMPLE', 'DUMMY', or 'test' as SAFE.

Treat active hardcoded cloud/API credentials, private keys, JWT secrets, credentialed DB URIs, and generic high-entropy tokens in auth/key/token-like

assignments as genuine secrets.

Focus on the target secret and its line first; use surrounding context only for disambiguation.

Appendix B. Parameter-Efficient Fine-Tuning (LoRA) Hyperparameters

Description: The exact parameters for hardware and software used in the local finetuning process for the Meta Llama 3 model on Apple MLX.

- **Model Name:** Meta Llama 3 8B Instruct
- **Execution Platform:** Apple MLX (mlx_lm)
- **Hardware Configuration:** Apple Silicon M-series (Unified Memory)
- **Quantization Scheme:** 4-bit (Reducing the 8B model to roughly 5.7GB memory usage)
- **Finetuning Technique:** Low Rank Adaptation (LoRA)
- **Attention Layer Targeted:** q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
- **LoRA r (Rank):** 8
- **LoRA α (Alpha):** 8
- **Number of Training Iterations (Max Steps):** 3,000 steps
- **Batch Size Per Device:** 1 (using 8 gradient accumulation steps to simulate batch size of 8)
- **Learning Rate:** 2e-4
- **Optimizer:** paged_adamw_8bit

Appendix C. System Data Structures and Output Contracts

Description: The expected output schema generated by the Llama 3 inference engine. The backend orchestration layer strictly validates this schema before passing the verdict to the frontend interface.

Standard LLM JSON Response Schema:

```
{  
  "is_genuine_secret": true,  
  "confidence_score": 0.98,  
}
```

```
"remediation_priority": "CRITICAL"  
}
```

Batch Evaluation JSON Response Schema (Optimized for CI/CD):

```
[  
  {  
    "line_number": 42,  
    "is_genuine_secret": false,  
    "confidence_score": 0.95,  
    "remediation_priority": "SAFE"  
  },  
  {  
    "line_number": 88,  
    "is_genuine_secret": true,  
    "confidence_score": 0.89,  
    "remediation_priority": "HIGH"  
  }  
]
```

8. REFLECTION OF THE TEAM MEMBERS ON THE PROJECT

8.1 Collective Team Learnings

In terms of engineering principles, we discovered that simply bolting a Large Language Model on top of our traditional static analysis pipeline is no panacea, as it presents us with as many new technical problems as it solves. However, we successfully gained experience with the integration of a deterministic heuristic filtering method (Regex) and non-deterministic semantic verification (LLM), demonstrating that context blindness can be effectively addressed even with legacy tools.

However, more importantly, we discovered what it takes to engineer systems within the Zero Trust framework. Although running our models locally on Apple silicon does guarantee full data privacy as our proprietary code does not leave the machine and gets processed using local LLMs without any calls to external cloud APIs, the cost of such an approach is extremely high in terms of computing overhead.

8.2 Individual Learnings

Dinanath Dash:

My main technical challenge was learning about the extremely unforgiving limitations of executing on locally installed AI hardware. Getting the inference script to run within Apple's Unified Memory while training an 8-billion parameter model forced an aggressive approach to 4-bit quantization and fine-tuned LoRA tuning in order to restore lost classification accuracy from the quantization step.

Apart from machine learning, I learned the complete engineering cycle necessary to turn the inference script in Python into a fully integrated end product. Constructing the front end, as well as successfully connecting it to the backend through machine learning orchestration, was an exercise in learning asynchronous state handling and managing the connection during lengthy LLM generation time.

Avignyan Sarangi:

I gained insight into the nuances of testing machine learning algorithms with regard to standard deterministic methods. By setting up and executing the CredData benchmark, and then examining the confusion matrices generated from them, I gained a clear understanding of how a simple tool such as Gitleaks attained a good precision rate but completely failed at recall rates.

What I learned from this project was how to interpret the significance of an F1-Score within the domain of security. I realized that in DevSecOps, a False Negative (failure to detect credentials) would always be worse than a False Positive (nuisance alarms). Striking the balance between our F1-Score of 87.23% and the inference lag posed by the model was my biggest technical lesson.

Satyajeet Satapathy:

One of my key learning outcomes was realizing the huge issue with "alert fatigue" in the modern software supply chain. Through framing the problem and conducting research in the DevSecOps field, I was able to learn first-hand why a high false-positive rate is so important for enterprise security teams because it causes developers to completely turn off their security tools.

Moreover, as a part of my project, it was essential for me to transform complicated raw data from the local MLX model, quantization algorithms, and LoRA parameters into academic language.

Ashutosh Raj:

I gained insights on how to ensure data validation in the engineering sense by integrating non-deterministic AI models into synchronous software workflows. The process of validating and implementing JSON schemas in respect to the outputs from Llama 3 made me understand how to avoid pipeline failures due to hallucinations or padding of LLM conversations.

Furthermore, ensuring that all the data from the entire project is accurate has been a huge lesson in itself. The processes of ensuring reproducibility of the metrics, standardization of the final thesis to adhere to IEEE standards as well as maintaining the less than 16% similarity in Turnitin were valuable lessons in technical compliance.

8.3 Strengths and Weaknesses of the Design Process

A major strength of our design phase was the empirical and cascading approach to architecture. In using Regex as a Stage-1 prefilter with a rigid approach, we prevented token exhaustion at its inception. Additionally, the deliberate constraint of the LLM's field of view to an AST micro-context with 11 lines is our greatest architectural success; it made our LLM solely dependent on variable semantics and helped to achieve our dominant 87.23% F1 Score.

A huge weakness in our engineering phase was our premature prioritization of detection accuracy without regard for inference latency. Through failing to engineer a batched pipeline from the start, we created a system that will take roughly 3.5 hours to process 1,000 hits of data. The biggest learning experience for this project in terms of DevSecOps has been that accuracy is not the only consideration; inference speed is equally important.

9. SIMILARITY REPORT